FraudArena

24-Hour Implementation Plan

Exact definitions, build order and checks for the Colossus finals

COLOSSUS 3.0 FINALS · 13 TO 14 NOVEMBER 2026

Version 2 · 12 November 2026 · everything built by H18, H18 to H24 for fixing and rehearsal

Numbers marked "target" or "illustrative" are to be replaced by values measured in our own runs.

Contents

#	Section
1	Goal, scope and the decisions that keep it buildable
2	Architecture
3	Repository and environment
4	Contracts: events, plans, rules, decisions, ledger records
5	Simulation core: clock, state, features, rule engine
6	Synthetic world and normal traffic
7	Attack playbooks and the attacker agent
8	Evaluation: metrics and the worst-case search
9	ML pipeline: data, splits, models, rule mining
10	Defender agent
11	Verifier: gates G1 to G6
12	Release, approval, rollback and the audit ledger
13	Database schema and API
14	Dashboard
15	Roles and the hour-by-hour schedule
16	Tests and the definition of done
17	Demo script, judge questions, risks and cut list
18	Tonight: before the event

How to read this. Sections 1 to 14 define *what* is built, exactly. Section 15 says *who builds what and when*. Anything stated as a number (thresholds, sizes) is a default we can change in one config file, but we agree it in hour 0 so all four people code against the same values.

1. Goal, scope and the decisions that keep it buildable

1.1 What the judges must see working

A mock bank under attack by an **attacker agent**; a rule set that misses an attack; a **defender agent** proposing a rule whose thresholds come from data; **six plain-code gates** that accept or reject it; a shadow run and a **human approval**; a new rule version going live; a **tamper-evident ledger** entry; and a chart of worst-case loss falling across rule versions while the false-alarm rate stays inside its budget. Every number shown comes from a run done at the event.

1.2 Hard constraints

- Everything built by **H18**. H18 to H24 is only for fixes, recording, rehearsal and rest. **Feature freeze at H18.**
- Four people, one owner per folder, three shared contracts agreed in H0 to H1.
- One command starts the whole system: `make up`. One command runs the demo round: `make demo`.
- The demo must survive no internet: all LLM answers used in the demo are cached on disk.

1.3 Design decisions (and why)

Decision	Choice	Reason
Where the simulation runs	One Python package `fa`, in-process. The bank, guard and features are plain Python objects, not separate services.	A round with hundreds of attack evaluations must run in seconds. Network hops between services would make the search too slow and add failure points.
Event log	PostgreSQL tables (events, decisions) written in batches; no Kafka.	Replay and audit need a durable log, not a stream. Kafka goes on the roadmap slide.
Rule language	Rules are JSON conditions (feature, operator, value), all must hold. No CEL, no Go.	Directly maps to decision-tree paths, trivially verifiable, trivially diffable.
Time	Simulated clock: every event has a timestamp; features use only earlier events.	A 3-day attack costs milliseconds; runs repeat exactly with a seed.
Reset between evaluations	**Snapshot copy** of the in-memory state (copy.deepcopy of a compact dict-of-arrays state taken after history load), restored before each evaluation.	Avoids reloading history each time. Target: restore under 50 ms.
LLM role	Writes attack plans and rule rationales as schema-checked JSON. Never sets numeric thresholds, never decides acceptance.	Answers the "why trust an LLM" question and keeps results repeatable.
Human approval	A real button in the dashboard, by a different person than the one who started the round (name recorded).	Maker-checker; recorded in the ledger.
Kubernetes, Kafka, Go, Z3, Neo4j	Roadmap only.	Not needed for the demo; each adds hours of risk.

1.4 Scope table

Tier	Contents	Rule
Must (by H18)	World, features, rule engine, 3 playbooks, attacker agent + search, evaluator, ML tree + XGBoost + mined rules, defender, gates G1 to G6, shadow, approval, rollback, ledger with verify and tamper demo, dashboard (5 views), tests, one-command start, measured numbers	All must be demonstrable from a clean start
Should (only after Gate 3)	ML detector under test (XGBoost scored payments attacked by the search), decision-latency benchmark	Cut first
Never in the 24h	PaySim/NFPC experiments, Kubernetes, Kafka, Z3, blockchain anchoring	Roadmap slide only

2. Architecture

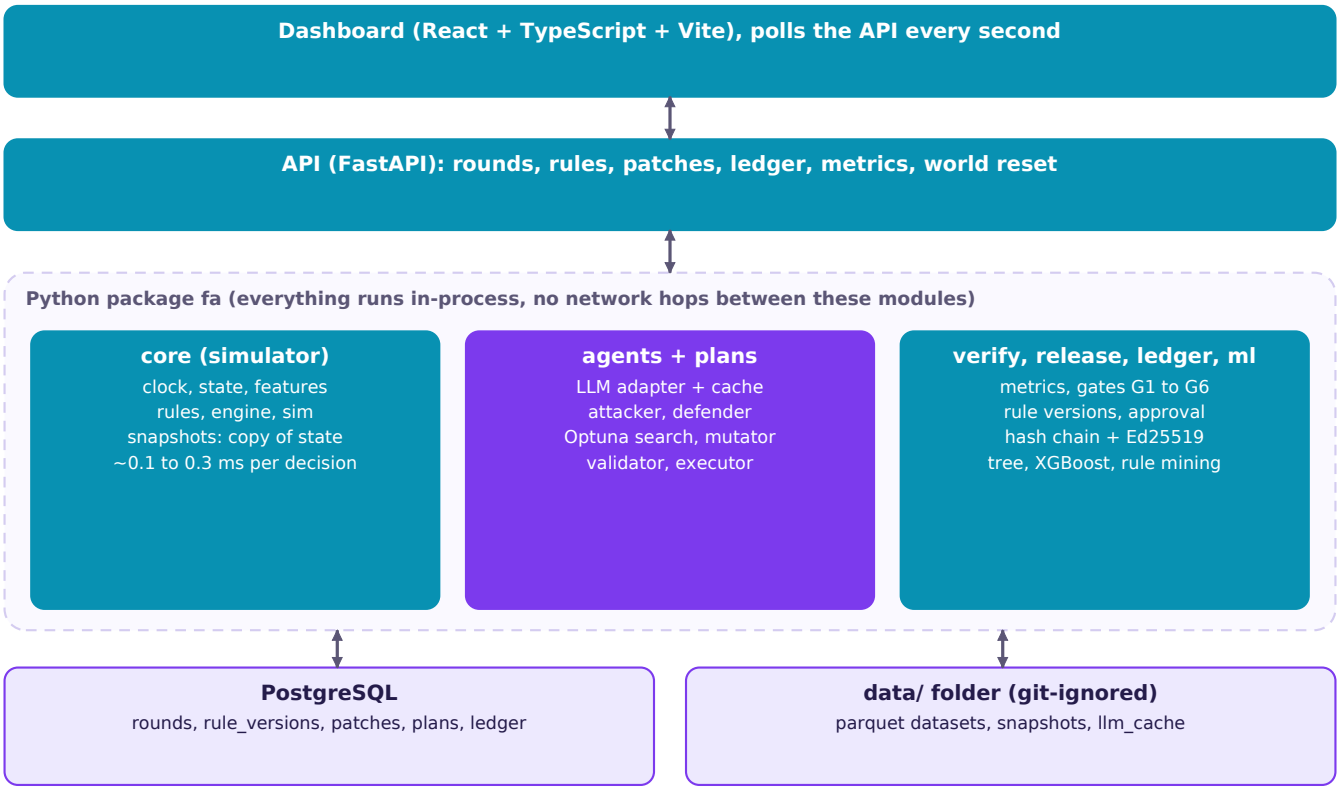


Figure 1. Purple = LLM agent code, teal = deterministic code, light = stored data. Only the API and the dashboard talk over the network; the simulation, agents, verifier and ledger are one Python process.

2.1 One round, in order

- **1 Select:** pick a playbook (P1, P2 or P3) and the active rule version.
- **2 Plan:** the attacker agent (LLM, cached) writes a base plan as JSON; the validator checks schema and caps.
- **3 Search:** Optuna tunes the plan parameters (default 150 trials); each trial restores the snapshot, runs the plan through the engine, scores loss.
- **4 Evaluate:** best plan = worst case for this rule version; the evaluator stores loss, decisions per step and the missed step.
- **5 Defend:** the defender analyses the failing step against normal traffic, picks thresholds from data, asks the LLM only for the rule text and rationale.
- **6 Verify:** gates G1 to G6 run in plain code; failure reasons go back to the defender (up to 3 attempts).
- **7 Release:** shadow run on validation traffic; a human approves in the dashboard; the version pointer moves; the ledger records it.
- **8 Repeat:** the search attacks the new version. The worst-case loss per version is the headline chart.

3. Repository and environment

```
fraudarena/
  fa/                     Python package (one owner per subfolder)
    core/                 clock.py state.py features.py rules.py engine.py sim.py snapshot.py   [Arena]
    world/                generator.py params.yaml normal.py                                    [Data/ML]
    plans/                schema.py validator.py playbooks.py executor.py mutator.py            [Lead]
    agents/               llm.py cache.py attacker.py defender.py search.py                     [Lead]
    ml/                   export.py split.py train.py mine_rules.py report.py                   [Data/ML]
    verify/               gates.py wilson.py regression.py golden.py shadow.py                  [Verifier]
    release/              versions.py approve.py rollback.py                                    [Verifier]
    ledger/               chain.py sign.py verify.py                                            [Verifier]
    orchestrator.py       round state machine                                                   [Lead]
  api/                    FastAPI app, routes, Pydantic models                                  [Arena]
  dashboard/              React + TypeScript + Vite                                             [Arena, Verifier]
  rules/                  v1.yaml (data-derived baseline), later versions written by release
  contracts/              JSON Schemas + fixture examples (all four agree)
  data/                   git-ignored: parquet, snapshots, llm_cache/
  tests/                  pytest
  docker-compose.yml  Makefile  .env.example  README.md
```

- **Versions:** Python 3.11, Node 20, PostgreSQL 16 (Docker), scikit-learn, xgboost, pandas, pyarrow, numpy, optuna, pydantic, fastapi, uvicorn, cryptography (Ed25519), pytest, hypothesis.
- **Make targets:** `up, down, world, round, demo, test, bench, reset`.
- **Config:** `fa/config.py` reads env and a single `params.yaml`; nothing is hard-coded in modules.
- **LLM:** one provider key in `.env`; `llm.py` exposes `complete_json(prompt, schema, key)`. Cache key = SHA-256 of (model, prompt, schema). On cache miss with no network it raises; the demo uses a pre-warmed cache.

4. Contracts

Written as JSON Schema in `contracts/` in H0 to H1 with example fixtures. A change needs all four people. Money is integer paise. Timestamps are simulation time in UTC.

4.1 Event

```
{"event_id":"e-0001842","ts":"2026-03-14T10:42:17Z",
 "type":"payment",            // login | add_payee | payment | cashout
 "account":"A-1042","device":"D-9","payee":"P-77",
 "amount":4800000,"channel":"upi"}
// Ground truth (is_fraud, playbook, plan_id) is stored in a separate table.
// The engine never sees it.
```

4.2 Attack plan

```
{"plan_id":"pl-P1-0007","playbook":"P1","seed":1337,
 "params":{"test_amt":100,"wait_s":600,"drain_mult":26,"splits":1,"split_gap_s":0},
 "steps":[
  {"t":0,   "actor":"victim","act":"login","device":"new"},
  {"t":180, "actor":"victim","act":"add_payee","payee":"mule"},
  {"t":480, "actor":"victim","act":"pay","payee":"mule","amount":100},
  {"t":1080,"actor":"victim","act":"pay","payee":"mule","amount":4800000},
  {"t":2400,"actor":"mule",  "act":"cashout","amount":4800000}]}
// Validator caps: sandbox roles only, at most 12 steps,
// horizon at most 72 h, total amount at most 200 x victim mean payment.
```

4.3 Rule and rule set

```
{"id":"R-018","action":"hold",
 "when":[{"f":"payee_age_h","op":"<","v":24},
         {"f":"amount_ratio_30d","op":">","v":8.0},
         {"f":"device_changed_24h","op":"==","v":true}],
 "rationale":"...","evidence":{"support":41,"precision":0.93,"fpr":0.0021},
 "source":"defender","added_in":"v2"}
// Rule set = ordered rules + version id.
// Decision: block if any block rule fires, else hold if any
// hold rule fires, else allow.
```

4.4 Decision and ledger record

```
Decision: {"event_id":"e-0001842","decision":"hold","fired":["R-018"],
           "version":"v2","latency_us":210}
Ledger:   {"seq":14,"ts":"...","action":"promote","from":"v1","to":"v2","patch_id":"pt-9",
           "gates":{"G1":"pass","G2":"pass","G3":"pass","G4":"pass","G5":"pass","G6":"pass"},
           "metrics":{"wcel_before":4800000,"wcel_after":1200000,
                      "hold_rate_before":0.008,"hold_rate_after":0.010},
           "maker":"defender-agent","checker":"reviewer-1",
           "prev_hash":"9f2c..","hash":"..","sig":".."}
```

5. Simulation core

5.1 State and the shared feature function

The state is a compact structure per account: a deque of (ts, amount) for outgoing payments in the last 30 days, a dict payee to first-seen ts, a dict device to first-seen ts, and sliding counters. Per recipient: deque of (ts, sender, amount) inflows and total outflow in the last hour. **One function** `compute_features(state, event)` returns the features below **from state before the event is applied**, then `apply(state, event)` updates state. The ML export and the live engine both call these two functions, so training and serving features cannot diverge.

Feature	Definition (all use only earlier events)
amount_ratio_30d	amount / mean outgoing payment of the account over the past 30 days; 1.0 if fewer than 3 payments
payee_age_h	hours since the payee was first added to the account (0 if the event is the add)
device_age_h / device_changed_24h	hours since the device was first seen on the account / whether a device first seen within 24 h is in use
n_pay_1h, n_pay_24h	count of outgoing payments in the last 1 h / 24 h
sum_1h, sum_24h	sum of outgoing amounts in the last 1 h / 24 h
sum_newpayee_1h	sum sent to payees younger than 24 h in the last hour
fan_out_1h	distinct payees paid in the last hour
r_fan_in_24h	distinct senders that paid the recipient in the last 24 h (0 if recipient unknown)
r_passthrough_1h	recipient outflow / recipient inflow over the last 1 h (0 if no inflow)
hour_of_day, night_flag	hour 0 to 23; true for 00:00 to 05:00

5.2 Engine

- `engine.decide(event)`: features, evaluate active rule set, return Decision, apply decision (allow updates balances and state; hold puts the payment in a pending list and does **not** count as money through; block rejects).
- **Loss through** = total fraudulent amount that was **allowed** and reached a mule account or cash-out. A held cash-out counts as zero.
- **Shadow:** `engine.decide_shadow(event, candidate)` evaluates a second rule set on the same features without applying anything.
- **Snapshot:** `snapshot.take(state)` / `restore`. Sandbox accounts (40 victims, 20 mules) are reset before each evaluation; normal accounts are restored from the same snapshot.
- **Performance target:** 0.1 to 0.3 ms per decision in pure Python; a 12-step plan evaluation under 20 ms including restore of sandbox accounts only (full restore only between rounds).

6. Synthetic world and normal traffic

Item	Specification (params.yaml)
Customers	1,000 in four segments: salaried 45%, student 20%, senior 15%, small business 20%. Each has 1 to 2 devices, 3 to 15 saved payees, a mean payment size (segment-specific log-normal), and a daily payment rate (segment-specific Poisson mean 0.6 to 2.5)
Normal payments	Amount ~ LogNormal(log(mean), 0.8). 85% to a saved payee, 15% to a new payee; time-of-day from a daytime-heavy curve; weekend factor 1.1 for retail, 0.5 for business. Recurring: rent monthly, bills monthly, subscriptions weekly or monthly
Look-alikes (important)	2% of normal payments are deliberately suspicious-looking but legitimate: first payment to a landlord (large, new payee), payment within 24 h of a phone change, festival purchase 5 to 10 times the mean, business paying a new supplier. Without these, rules trivially separate fraud
History length	30 days history (state warm-up), 7 days validation traffic, 7 days holdout traffic (reported numbers only)
Volume	About 1,000 x 1.4 x 44 days, roughly 60,000 normal payments; deterministic from SEED
Sandbox accounts	40 victim accounts (typical segment mix, excluded from normal traffic) and 20 mule accounts (look normal until used). Attacks only touch these
Determinism	`numpy.random.default_rng(seed)` everywhere; `make world SEED=42` twice gives identical parquet hashes (test)

7. Attack playbooks and the attacker agent

Playbook	Story	Searchable parameters (range)
P1 takeover and drain	new-device login, add payee, small test payment, wait, large payment (optionally split), cash-out	test_amt 1 to 500; wait_s 60 to 43,200; drain_mult 3 to 60 x victim mean; splits 1 to 6; split_gap_s 30 to 7,200; device_age_h 0 to 12
P2 mule chain	victim pays mule 1, then mule 2, then mule 3, then cash-out; splits and delays per hop	hops 1 to 3; split_k 1 to 4; hop_delay_s 30 to 21,600; amount_mult 3 to 40
P3 structuring	many payments just under a limit (limit L = 50,000 paise units configured) across payees and time	size_frac 0.6 to 0.99 of L; n_payees 1 to 8; spacing_s 60 to 14,400; n_payments 3 to 12

Attacker agent, step by step

- **Inputs:** playbook description and parameter ranges; the outcome (allow/hold/block per step) of earlier trials. It never sees rule text (black box).
- **LLM call** returns a base plan in the schema of 4.2 within the ranges. Prompt and schema hash form the cache key. Temperature 0 for the demo.
- **Validator** rejects anything outside caps (sandbox roles, at most 12 steps, 72 h, amount cap). Rejections are logged and the LLM is asked again (max 2).
- **Search (Optuna TPE, 150 trials per playbook per round, fixed seed):** objective = maximise loss through; ties broken by fewer steps and less waiting time. Random search with the same budget is the baseline shown on the chart.
- **Mutator (for gate G4):** deterministic operators with held-out seeds: scale amount by 0.5 to 1.5, shift waits by 0.5 to 2 times, split a payment into 2 to 4 parts, swap which payee, change device age, insert 1 to 3 decoy normal payments.

8. Evaluation: metrics and the worst-case search

Metric	Exact definition
Loss through	sum of fraudulent amounts allowed and cashed out (paise)
Attack success	loss through is at least 50% of the plan's intended total
WCEL	worst-case extractable loss: maximum loss through found by the 150-trial search for a playbook against a rule version; reported per playbook and as the sum
Hold rate / block rate	held (blocked) normal payments / all normal payments, on validation or holdout traffic; 95% Wilson interval: $\hat{p}=k/n$, $z=1.96$, centre=$(\hat{p}+z^2/2n)/(1+z^2/n)$, half-width=$z\cdot\sqrt{\hat{p}(1-\hat{p})/n+z^2/4n^2}/(1+z^2/n)$
Detection delay	simulated seconds from first attack step to first hold or block
Decision latency	p50/p95/p99 of engine.decide in a 50,000-event replay on the demo laptop (one run, reported as measured)

Headline chart: WCEL per playbook for versions v1, v2, v3, v4 (bars) with hold rate and its interval for each version (line). If WCEL does not fall, that is shown honestly with the reason.

9. ML pipeline

9.1 Purpose and rule

Rules and the baseline rule set must come from data and be measured on data the model has not seen. The LLM never sets a threshold. ML work is **training and mining**; it is not a separate service.

9.2 Step by step (owner: Data and ML)

- **Export (H4 to H8):** `ml/export.py` replays the world through `compute_features/apply` and writes parquet with one row per outgoing payment: features (section 5.1), account, ts, plus labels `is_fraud, playbook, plan_id` stored in **separate columns that are dropped before training** (asserted in code).
- **Attack rows:** run each playbook with random parameters (not the searched worst case): 300 plans per playbook against a permissive rule set (everything allowed) so every step produces features. Label every payment inside a plan as fraud.
- **Class balance:** training set may be weighted; **test sets keep fraud near 1%** of rows by subsampling attack payments, so precision numbers are honest.
- **Splits:** by time (train = days 1 to 30, validation = days 31 to 37, test = days 38 to 44) and by account (a sandbox victim or mule appears in only one split). Plus **leave-one-playbook-out**: train on two playbooks, test on the third.
- **Models:** DecisionTreeClassifier(max_depth=4, min_samples_leaf=30, class_weight='balanced', random_state=7) and XGBClassifier(n_estimators=200, max_depth=4, learning_rate=0.1, scale_pos_weight=neg/pos, random_state=7). No tuning.
- **Metrics:** precision, recall, false-alarm rate (on normal), PR-AUC, precision at 100, each for random split, held-out playbook and test traffic. Report a results table; any score above 0.99 is investigated as a bug before it is shown.
- **Mine rules (H8 to H11):** walk the tree; every leaf with fraud share at least 0.5 and support at least 30 becomes a rule (the path's conditions, merged per feature to the tightest bound). Compute support, precision and false-alarm rate on validation; keep rules with false-alarm rate at most 0.5% each. The kept set is **rule set v1** (the baseline). It is intentionally imperfect so the attacker finds gaps.
- **Thresholds for the defender:** `mine_rules.candidates(failing_step)` returns single and pair predicates ranked by (stops the step, lowest added false-alarm rate on validation), with evidence. The defender may only choose from these.

9.3 Leakage checklist (a second person signs it off before any number goes on a slide)

- No feature is computed using an event at or after the one being scored.
- Label columns and plan IDs are not in the feature matrix (assert on column names).

- Thresholds chosen on validation only; test touched once.
- Same account never in two splits.
- Fixed seeds written in the report.

9.4 If ML runs late

Fallback in code, switchable by `ML_MODE=percentile`: candidates are single predicates at the 99th, 99.5th and 99.9th percentile of the feature among normal payments. Still data-derived. We state in the pitch which mode produced the shown rules.

10. Defender agent

- **Input:** the failing trace (per-step features and decisions), current rule set, and `mine_rules.candidates()` for the missed step.
- **Selection (code):** choose the candidate with the largest stopped loss subject to added validation false-alarm rate at most 0.5 percentage points. Thresholds are copied from the candidate.
- **LLM call (cached):** input = chosen conditions + evidence; output schema = `{id, rationale, expected_effect}`. The LLM cannot change conditions; the code assembles the final rule JSON.
- **Retries:** if a gate fails, the failure reason and the next-best candidate are used; at most 3 attempts. If none passes, the round ends with **NO_SAFE_PATCH**, shown on the dashboard as a valid outcome.
- **Fuzz test:** the rule assembler never emits an unknown feature name or operator.

11. Verifier: gates G1 to G6

Gate	Check	Pass rule (defaults in params.yaml)	Evidence stored
G1 valid	JSON schema passes; features in the allowed list; operators in {<,<=,>,>=,==}; at most 4 conditions; not identical to an existing rule	all true	validation log
G2 stops the failing attack	replay the failing plan (same seed) on candidate set	loss through = 0 and decision at or before cash-out is hold or block	per-step decisions
G3 no regression	replay every plan in the regression set (all earlier plans that once got loss > 0, with their seeds)	all still loss = 0	per-plan table
G4 variants	50 mutated variants of the failing plan (held-out seeds)	at least 80% have loss = 0 (actual rate recorded)	per-variant results
G5 friction	replay 7 days of validation normal traffic	Wilson 95% upper bound of hold+block rate at most 2.0%, and at most 0.5 percentage points above the active set	counts, interval
G6 golden benign	200 curated normal-but-unusual cases (rent to new landlord, post phone change, festival purchase, new supplier)	all 200 still allowed	case list

- Gates are pure functions: `gate(candidate, context) -> {pass, reason, evidence}`; deterministic; no LLM.
- **Six unit tests** prove each gate rejects a deliberately bad patch: invalid feature (G1), irrelevant rule (G2), a rule that undoes an old fix (G3), an over-fitted rule that stops only one amount (G4), an over-broad rule (G5), a rule that blocks rent to a new landlord (G6).
- Shadow results (new holds, lost catches) are computed by the same code and shown in the Patch review view.

12. Release, approval, rollback and the audit ledger

State	What happens	Next
PROPOSED	defender submits a patch	VERIFIED if G1 to G6 pass, else REJECTED with reasons
VERIFIED	shadow run on 7 days validation + regression set, differences reported	AWAITING_APPROVAL (automatic)
AWAITING_APPROVAL	dashboard shows diff, evidence, shadow results; a checker (not the round starter) approves or rejects, name recorded	ACTIVE or REJECTED
ACTIVE	version pointer moves atomically; ledger entry written	ROLLED_BACK possible
ROLLED_BACK	previous version restored; ledger entry written	terminal

Ledger

- Append-only table. `hash = SHA256(canonical_json(record without hash and sig) + prev_hash)`; canonical JSON = sorted keys, no spaces. `sig = Ed25519(private_key, hash)`; the demo key is generated at first start and stored under `data/`.
- **verify()** recomputes the chain and checks every signature; returns the first broken sequence number.
- **Tamper demo:** a button copies the table to a scratch table, edits one field, runs verify on the scratch copy, and shows the failing row. The real table is never modified.
- Genesis record has prev_hash of 64 zeros.

13. Database schema and API

```
rounds(id, started_at, playbook, base_version, status, wcel_before, wcel_after, note)
plans(id, round_id, playbook, seed, params_json, loss, decisions_json, is_best)
rule_versions(version, parent, created_at, rules_json, source)
patches(id, round_id, rule_json, state, gates_json, shadow_json, maker, checker, created_at)
events(id, run_id, ts, account, type, amount, decision, version)      -- written in batches of 1,000
ledger(seq, ts, action, payload_json, prev_hash, hash, sig)
metrics(run_id, version, playbook, wcel, hold_rate, hold_lo, hold_hi, p50_us, p99_us)
```

Endpoint	Purpose
POST /api/rounds · GET /api/rounds/{id}	start a round; poll state, current step, plans, patch, gates
GET /api/versions · GET /api/versions/{v}	rule versions and diffs
POST /api/patches/{id}/approve · /reject · POST /api/rollback	maker-checker actions (body has the checker name)
GET /api/metrics · GET /api/ledger · GET /api/ledger/verify · POST /api/ledger/tamper-demo	evolution chart data, ledger table, chain check
POST /api/world/reset	restore the snapshot and set the baseline version
GET /healthz	health

14. Dashboard (five views, large type, high contrast)

View	Shows	Built by
1 Arena	active version, WCEL per playbook, hold rate with interval, buttons: Run round, Reset	Arena (H8 to H11)
2 Round	attack timeline: each step with decision and key features; missed step highlighted; plan parameters	Arena
3 Patch	rule diff, rationale, evidence, G1 to G6 checklist, shadow differences, Approve and Reject	Verifier (H8 to H11)
4 Evolution	WCEL per playbook by version, hold rate with interval, random-search baseline	Data/ML (data) + Arena (chart)
5 Ledger	table of entries, Verify, Tamper demo, Rollback	Verifier

React + TypeScript + Vite, a chart library (Recharts or ECharts), polling every 1 s while a round is running. No dense tables on the main views. A pre-recorded fallback video and screenshots of each view are prepared in H18 to H20.

15. Roles and the hour-by-hour schedule

Role	Owns
Lead	contracts, plans/, agents/, orchestrator.py, integration, demo
Arena	core/, api/, docker-compose, dashboard views 1 and 2
Data and ML	world/, ml/, evaluator metrics, experiments, charts
Verifier and release	verify/, release/, ledger/, tests, dashboard views 3 and 5

Assign by strengths; the Data and ML person must not also be the integration owner. H0 = the moment coding starts.

Hours	Lead	Arena	Data/ML	Verifier/release
H0 to H1	Contracts v1 (section 4), repo, Makefile, LLM key test	docker-compose with Postgres, .env, health endpoint	params.yaml draft, feature list reviewed	Ledger schema, rule_versions schema
H1 to H4	llm.py + cache; schema.py, validator.py; playbooks P1 to P3 as parameter ranges; executor	clock, state, features.py, rules.py, engine.py, snapshot.py, tests for each feature on hand-built cases	generator.py: customers, devices, payees, normal traffic, look-alikes; seed test	wilson.py, chain.py, sign.py, verify.py with tests; versions.py
Gate 1 (H4)	a stub plan runs through engine and returns decisions	engine passes feature tests; 50,000-event replay runs	world hash is stable; parquet written	ledger tamper test passes
H4 to H8	attacker.py, search.py (Optuna), mutator.py; orchestrator skeleton	API routes for rounds, versions, metrics; sim.py wiring; shadow decide	ml/export.py with shared features; split.py; train.py (tree, XGBoost); first results table	gates G2 to G6 (G1 done), regression set store, golden set (200 cases) built with Data/ML
Gate 2 (H8)	plan to best plan under 20 ms per trial	round state is queryable over API	results table exists, leakage checklist drafted	all six gates pass their unit tests
H8 to H11	defender.py; orchestrator runs P1 end to end	dashboard shell, Arena and Round views	mine_rules.py, rule set v1 from data, candidates() for defender	approve, reject, rollback; ledger endpoints; Patch view and Ledger view

Hours	Lead	Arena	Data/ML	Verifier/release
H11 to H13	**integration sprint, everybody:** one full round on P1 (attack, loss, patch, gates, shadow, approval, new version, ledger)	fixes	fixes	fixes
Gate 3 (H13)	one full round works end to end, even if ugly. If not, apply the cut list immediately			
H13 to H16	P2 and P3 in the loop; rounds v1 to v4; ML_MODE fallback	make up, make demo, one-command start; Evolution chart	experiment runner, charts, random-search baseline, decision-latency run, leakage sign-off	pytest suite for gates, ledger, determinism; LLM-offline path
Gate 4 (H16)	three playbooks, five views, ledger verify and tamper demo, one command start			
H16 to H18	three full demo runs from a clean start; warm the LLM cache; README with built vs roadmap	clean-clone test on a second laptop	final numbers into a results file	final test run
H18	**FEATURE FREEZE**			

H18 to H24

- **H18 to H20:** fixes only; record the backup video; screenshots; put measured numbers in the slides.
- **H20 to H22:** rehearse the six-minute demo three times; practise the judge questions; test failures (LLM off, Wi-Fi off, reset to clean).
- **H22 to H24:** rest in turns of about 90 minutes, two people at a time; stage the demo laptop and a backup.

16. Tests and the definition of done

Level	What	Tool
Unit	each feature on hand-built event sequences; rule engine precedence; Wilson interval against known values; each gate (good and bad patch); ledger hash and signature	pytest
Property	mutator output always passes the validator; validator rejects malformed plans; assembler never emits unknown features	hypothesis
Determinism	same SEED gives the same world hash; same plan and seed gives the same decisions	pytest
End to end	one P1 round using cached LLM output ends with an ACTIVE version and a valid ledger	pytest + docker compose
Fault	LLM off with warm cache still runs; unknown cache miss fails with a clear message	manual script

Done by H18 (checklist)

- `make up` from a clean clone starts everything; `make demo` runs a round.
- P1, P2, P3 all run; the search beats random search on at least one (shown on the chart).
- Gates accept a good patch and reject six bad ones.
- Shadow, approval (by a different person) and rollback work.
- Ledger verifies; the tamper demo fails at the edited row.
- Five dashboard views work; polling shows live progress.
- ML results table (tree, XGBoost; random split, held-out playbook), mined rules with evidence, leakage sign-off.
- WCEL by version, hold rate with interval, decision-latency numbers, all measured.

- LLM cache warm; backup video recorded; README states what is built vs roadmap.

17. Demo, judge questions, risks and cut list

Time	On screen	Say
0:00	problem slide, one number	fraud is mostly social engineering; banks find rule gaps after losses
0:30	Arena view: v1, WCEL, hold rate	this is how much an attacker can still take today
1:15	Round view: attack trace	a ₹1 test, then a big drain, slips past v1
2:30	Patch view: diff, gates turn green	AI drafts; thresholds come from data; plain code decides
3:30	Approve (second person), v2 live	maker-checker; shadow run shown
4:15	Evolution chart	worst-case loss falls, customer friction stays inside budget
4:45	Ledger: tamper demo	every change is auditable
5:15	numbers + roadmap	latency measured; Kubernetes, Kafka, graph view next

Judge question	Answer
How is this different from MuleHunter?	A detector scores accounts; we attack any detector or rule set and prove the fix before release. A detector can be the system under test.
Why trust an LLM with rules?	We do not. Thresholds come from data analysis, six plain-code gates verify, shadow and human approval follow, and the ledger records everything.
Is the data real?	Synthetic and labelled; we claim relative robustness gains, not real-bank accuracy.
Could the defender overfit?	G3 replays all past attacks, G4 uses unseen variants, G6 protects normal cases, holdout traffic is never used for tuning.
What did you build vs plan?	The README table: built and measured, built, roadmap.

Risk	Mitigation
Scope too large	Gate 3 at H13; cut list below; no new features after H18
LLM slow or offline	schema-checked JSON, disk cache, pre-warmed demo cache, percentile fallback
Integration surprises	contracts first, stubs from H1, integration sprint H11 to H13
ML late	ML_MODE=percentile; never claim a model we did not train
Docker or Wi-Fi trouble	test on every laptop tonight; one reference laptop plus a backup; offline demo
Demo fails live	backup video, screenshots, make reset, rehearsed three times

Cut order: (at Gate 2) P3, search tuner, XGBoost; (at Gate 3) P2, evolution polish, fewer golden cases; (at Gate 4) tamper button polish, extra experiments; any time: ML detector under test, latency benchmark beyond one run. **Never cut:** the end-to-end round, the six gates, the ledger, the dashboard, honest numbers.

18. Tonight (12 Nov) before the event

- Confirm with the organisers what may be brought or prepared; write down the answer.
- Agree the three contracts (section 4) and the params.yaml defaults on a call; assign the four roles.
- One LLM key with credit; test `complete_json` and the cache on every laptop.
- Docker and Docker Compose work on every laptop (WSL2 on Windows); pull Postgres 16, Python 3.11 and Node 20 images and all pip and npm packages while online.
- Create the private Git repository, add all four, push the skeleton from section 3.
- Charge laptops; chargers, extension lead, phone hotspot; print Section 16 checklist and the leakage checklist.
- Decide which laptop runs the demo; sleep.

Honest scope note. This is a lot for 18 hours. It works because the simulation is one in-process package, the rule language is tiny, and the LLM and ML parts are narrow. If Gate 3 slips, cut by the table in section 17 without debate.